

Praktikum 4: Binäre Suchbäume (BST)

Binäre Suchbäume erlauben das effiziente Suchen, Einfügen und Löschen von Schlüsseln in $O(\log n)$ - vorausgesetzt, der Baum ist ausgewogen. Ob das tatsächlich der Fall ist, ist keineswegs garantiert. In diesem Praktikum untersuchen Sie dieses Verhalten experimentell, lernen ein Visualisierungswerkzeug kennen und erweitern die vorhandene Implementierung um das Löschen im AVL-Baum.

Die Klassen `BinaryTree`, `BinaryTreeNode`, `AVLTree` und `AVLTreeNode` aus der Vorlesung stehen Ihnen vollständig zur Verfügung.

Visualisierung mit Graphviz

Graphviz (<https://graphviz.org/>) ist ein quelloffenes Werkzeug zur automatischen Berechnung von Graphen- und Baumlayouts. Aus einer textuellen Beschreibung (z.B. `graph.gv`) in der DOT-Sprache erzeugt es unter anderem PDF-Dateien:

```
dot -Tpdf graph.gv > graph.pdf
```

Installieren Sie Graphviz auf Ihrem Rechner.

Laden Sie anschließend die Folge `seq0.txt` in einen `BinaryTree` und rufen Sie `graph_traversal()` auf. Öffnen Sie das erzeugte PDF und beantworten Sie:

- Welcher Knoten ist die Wurzel?
- Wie viele Blattknoten hat der Baum?
- Wie groß ist die Höhe des Baums?

Traversierungen

Führen Sie auf demselben Baum die folgenden Traversierungen aus und vergleichen Sie die Ausgaben:

- `in_order_traversal`
- `level_order_traversal`

Beantworten Sie:

- Welche Traversierung gibt die Schlüssel in aufsteigend sortierter Reihenfolge aus, und warum ist das bei einem BST zwangsläufig so?
- Was zeigt `level_order_traversal`, das `in_order_traversal` nicht erkennen lässt?
- Schauen Sie sich den Quellcode von `graph_traversal` an. Welche Traversierung wird intern genutzt, und wozu dienen die Parameter `level` und `line`, die an den Callback übergeben werden?

Entartung des BST

Fügen Sie die Folge [1, 2, 3, 4, 5, 6, 7, 8] in einen neuen **BinaryTree** ein und visualisieren Sie das Ergebnis mit Graphviz.

Vergleichen Sie den Baum mit dem aus Aufgabe 1 und beantworten Sie:

- Wie viele Vergleiche benötigt **search(8)** in diesem Baum?
- Wie viele Vergleiche wären es bei einem vollständig ausgewogenen Baum mit 8 Knoten?
- Welche Eingabereihenfolge würde beim BST stets einen ausgewogenen Baum erzeugen?
- Was müsste eine Datenstruktur (wie z.B. ein AVL-Baum) garantieren, um dieses Problem grundsätzlich zu lösen?

Löschen im AVL-Baum

Bauen Sie einen AVL-Baum mit den Werten [10, 5, 15, 3, 7, 12, 20, 1, 4, 6, 8] auf und löschen Sie jeweils einen Knoten für alle drei Fälle:

- Löschen eines Blattknotens
- Löschen eines Knotens mit genau einem Kind
- Löschen eines Knotens mit zwei Kindern

Visualisieren Sie den Baum jeweils vor und nach dem Löschen mit Graphviz und überprüfen Sie, dass der AVL-Baum nach jeder Operation korrekt ausgewogen ist. Beantworten Sie anhand des Quellcodes von **delete** in **AVLTree**:

- Nach dem Löschen wird **node.parent = parent** neu gesetzt. Warum ist das notwendig? Was würde ohne diese Zeile passieren?
- Die Rebalancierung beginnt mit **balance(parent)**, nicht mit **balance(node)**. Warum?

Komplexitätsanalyse

Leiten Sie die Laufzeit-Komplexität von **search** für die folgenden drei Fälle theoretisch her:

- AVL-Baum: Formulieren Sie die Rekurrenz für die Suche und lösen Sie sie mit dem Master-Theorem.
- BST (Worst Case - sortierte Eingabe): Formulieren Sie die Rekurrenz und lösen Sie sie.
- BST (Zufällige Eingabe): Geben Sie die erwartete Höhe an und begründen Sie sie intuitiv.

Messen Sie anschließend die Anzahl der Vergleiche bei **search** für $n \in \{10, 100, 500\}$ und tragen Sie die Werte in eine Tabelle ein:

n	BST zufällig	BST sortiert	AVL sortiert
10			
100			
500			

Stimmen die gemessenen Werte mit den theoretisch hergeleiteten Komplexitätsklassen überein?

Musterlösung Praktikum 4: Binäre Suchbäume (BST)

1 Visualisierung mit Graphviz

1.1 Aufbau und Visualisierung

Der BST wird mit den Werten aus `seq0.txt` befüllt und anschließend durch `graph.traversal()` als DOT-Datei exportiert:

```
values = Array.from_file('data/seq0.txt', ctx)
tree = BinaryTree(ctx)
for cell in values:
    tree.insert(cell.value)
tree.graph_traversal() # erzeugt BinaryTree.gv und BinaryTree.pdf
```

Das erzeugte PDF lässt sich mit dem Befehl

```
dot -Tpdf graph.gv > graph.pdf
```

aus einer bestehenden DOT-Datei regenerieren.

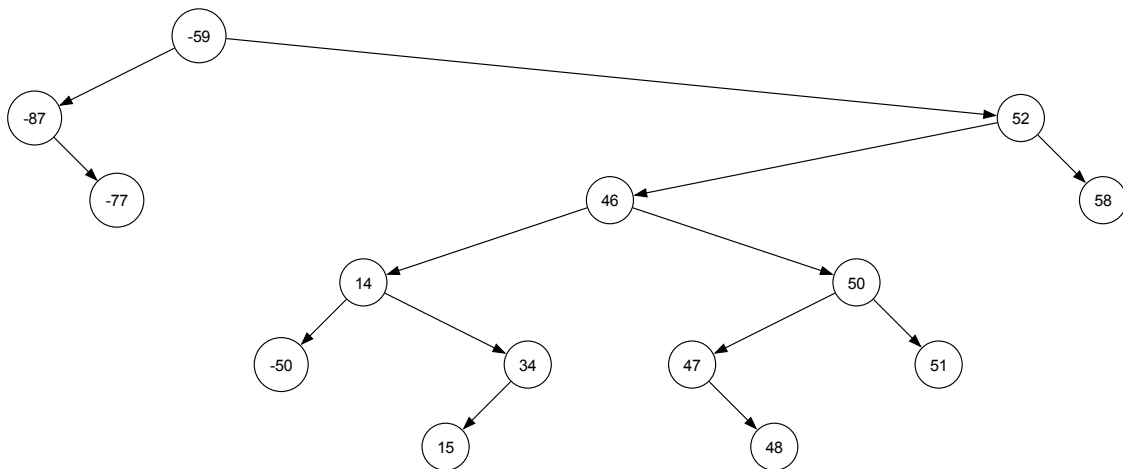


Abbildung 1: BST nach Einfügen der Werte aus `seq0.txt`

1.2 Beobachtungen am erzeugten Baum

- **Wurzel:** Das erste eingefügte Element wird immer die Wurzel, da der Baum anfangs leer ist. Bei `seq0.txt` ist das `-59`.
- **Blattknoten:** Knoten ohne Kinder. Der Baum besitzt `{-77, -50, 15, 48, 51, 58}`, also **6** Blätter.
- **Höhe:** Längster Pfad von der Wurzel zu einem Blatt. Mit 14 Elementen in zufälliger Reihenfolge ergibt sich Höhe **6**.

2 Traversierungen

2.1 In-Order-Traversal

```
tree.in_order_traversal(lambda node: print(node.value, end=" "))
# Ausgabe: -87 -77 -59 -50 14 15 34 46 47 48 50 51 52 58
```

Warum liefert In-Order eine aufsteigend sortierte Folge?

Die BST-Eigenschaft garantiert: linker Teilbaum $<$ Wurzel $<$ rechter Teilbaum. In-Order besucht genau in der Reihenfolge *links* $\rightarrow$ *Wurzel* $\rightarrow$ *rechts*. Rekursiv angewendet ergibt das eine streng aufsteigende Sortierung aller Schlüssel – bei einem BST ist In-Order-Traversal damit äquivalent zu einer vollständigen Sortierung der Elemente.

Widerspruchsüberlegung: Angenommen, eine größere Zahl b erscheint in der In-Order-Ausgabe *vor* einer kleineren Zahl a (d.h. $a < b$, aber b wird früher besucht). Da In-Order zuerst den gesamten linken Teilbaum ausgibt, bedeutet das: b liegt im linken Teilbaum eines gemeinsamen Vorfahren v , während a rechts davon steht – oder b ist Vorfahre von a und a liegt in seinem rechten Teilbaum. In beiden Fällen würde die BST-Eigenschaft verletzt ($b \geq v > a$ bzw. $b < a$). **Widerspruch.** Also kann keine größere Zahl früher ausgegeben werden als eine kleinere.

2.2 Level-Order-Traversal

```
tree.level_order_traversal(lambda node, level: print(f"Ebene {level}: {node.value}"))
```

Funktionsweise

Im Gegensatz zur rekursiven In-Order-Traversal arbeitet `level_order_traversal` mit einer *Warteschlange* (Liste, aus der vorn entnommen und hinten angefügt wird):

1. Zu Beginn enthält die Warteschlange nur die Wurzel.
2. Solange die Warteschlange nicht leer ist:
 - (a) Entnehme den *vordersten* Knoten und gib ihn aus.
 - (b) Füge seine Kinder *hinten* an die Warteschlange an.

Das Entscheidende ist das FIFO-Prinzip (First In, First Out): alle Knoten einer Ebene werden eingereiht, bevor der erste Knoten der nächsten Ebene entnommen wird. Damit ist garantiert, dass die Ausgabe Ebene für Ebene erfolgt – Ebene 0 (Wurzel), dann Ebene 1 (Kinder der Wurzel), dann Ebene 2 usw.

Was zeigt Level-Order, was In-Order nicht zeigt?

Level-Order zeigt, auf welcher Tiefenebene sich jeder Knoten befindet, und damit die *tatsächliche Baumstruktur*. Zwei BSTs mit identischen Schlüsseln liefern immer dieselbe In-Order-Ausgabe, aber unterschiedliche Level-Order-Ausgaben, wenn ihre Struktur verschieden ist.

2.3 Interner Aufbau von `graph_traversal`

`graph_traversal` nutzt intern `tree_structure_traversal` – eine In-Order-Traversal, die zusätzlich die Parameter `level` und `line` mitliefert:

- `level` (Tiefe) $\rightarrow$ y -Koordinate im Graphviz-Layout: je tiefer, desto weiter unten.
- `line` (In-Order-Position) $\rightarrow$ x -Koordinate: die aufsteigende In-Order-Nummerierung ergibt automatisch eine kollisionsfreie horizontale Anordnung der Knoten.

3 Entartung des BST

3.1 Sortierte Einfügereihenfolge

```
tree_sorted = BinaryTree(ctx)
for v in range(1, 9):      # [1, 2, 3, 4, 5, 6, 7, 8]
    tree_sorted.insert(v)
tree_sorted.graph_traversal()  # sichtbare Kette nach rechts
```

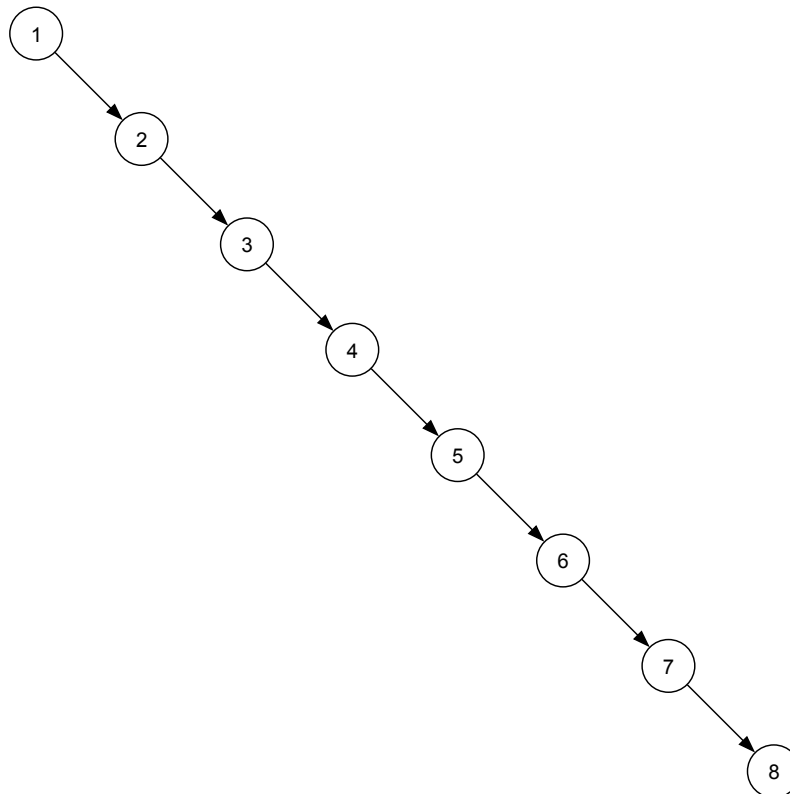


Abbildung 2: BST nach Einfügen von [1, 2, 3, 4, 5, 6, 7, 8] – entartet zur verketteten Liste

Da jede neue Zahl größer ist als alle vorherigen, wird sie immer als rechtes Kind des aktuell rechtesten Knotens eingefügt. Der Baum entartet zur *verketteten Liste* mit Höhe n .

3.2 Vergleiche für `search(8)`

- **Entarteter Baum (sortierte Eingabe):** 16 Vergleiche. Jeder der 8 Knoten erfordert zwei Vergleiche (`value < node?` und `value > node?`). Das entspricht einer linearen Suche: $\mathcal{O}(n)$.
- **Ausgewogener Baum (Höhe 4):** 8 Vergleiche. Eingabereihenfolge z. B. [4, 2, 6, 1, 3, 5, 7, 8] (Mitte zuerst).

3.3 Analysefragen

Welche Eingabereihenfolge erzeugt stets einen ausgewogenen BST?

Median-First: stets den Median der aktuellen Teilfolge zuerst einfügen (Divide-and-Conquer über

die Schlüsselmenge). Das setzt voraus, dass alle Elemente vorab bekannt sind – bei dynamischen Einfügungen ist das nicht möglich.

Was muss eine Datenstruktur für eine akzeptable Laufzeit garantieren?

Sie muss die Baumhöhe nach jeder Einfüge- oder Löschoperation *automatisch* auf $\mathcal{O}(\log n)$ begrenzen – unabhängig von der Eingabereihenfolge. AVL-Bäume erreichen dies durch Überprüfung des Balance-Faktors und gezielte Rotationen nach jeder Operation.

4 Löschen im AVL-Baum

4.1 Aufbau des Ausgangsbaums

```
tree = AVLTree(ctx)
for v in [10, 5, 15, 3, 7, 12, 20, 1, 4, 6, 8]:
    tree.insert(v)
```

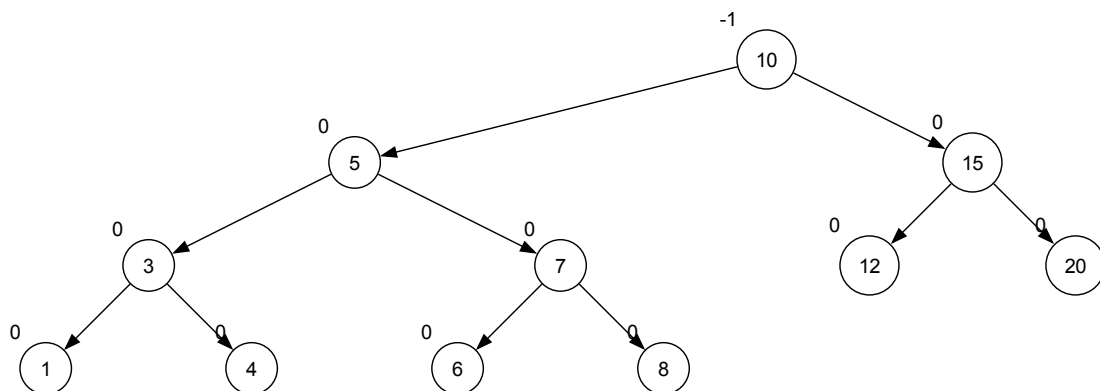


Abbildung 3: AVL-Baum nach dem Einfügen von [10, 5, 15, 3, 7, 12, 20, 1, 4, 6, 8]

4.2 Die drei Löschfälle

4.2.1 Fall 1: Blattknoten löschen

`tree.delete(1)` – Knoten 1 hat keine Kinder und wird einfach entfernt. Der Elternknoten (3) erhält `left = None`; anschließend wird von 3 aufwärts rebalanciert.

4.2.2 Fall 2: Knoten mit genau einem Kind löschen

`tree.delete(3)` – nach dem Löschen von 1 hat Knoten 3 nur noch Kind 4. Kind 4 rückt an Knoten 3's Stelle; der Elternknoten wird aktualisiert und rebalanciert.

4.2.3 Fall 3: Knoten mit zwei Kindern löschen

`tree.delete(10)` – Knoten 10 hat linkes Kind 5 und rechtes Kind 15. Der *Inorder-Nachfolger* (kleinstes Element im rechten Teilbaum) ist 12. Sein Wert überschreibt den von 10; anschließend wird Knoten 12 an seiner ursprünglichen Position gelöscht und von dort aufwärts rebalanciert.

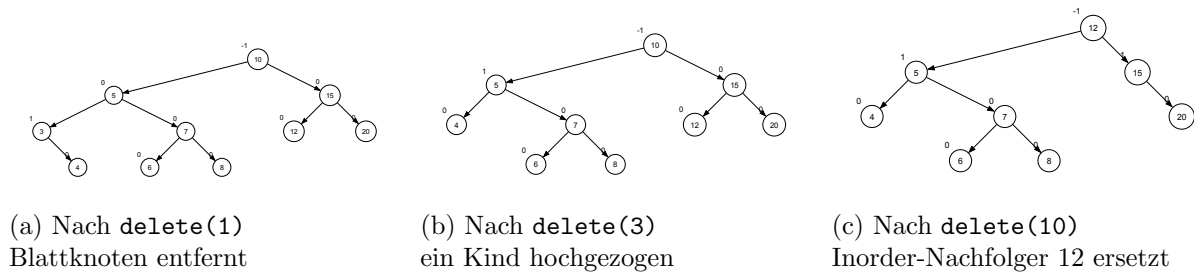


Abbildung 4: AVL-Baum nach jedem der drei Löschfälle (jeweils rebalanciert)

4.3 Verständnisfragen zum Quellcode von `delete` in `AVLTree`

Warum muss `node.parent = parent` nach dem Löschen neu gesetzt werden?

`BinaryTree` kennt keine `parent`-Zeiger – `delete()` gibt lediglich (`Nachfolger`, `Elternknoten`) als Tupel zurück. `AVLTree` pflegt `parent`-Zeiger, weil `balance()` sie benötigt, um den Pfad von einem Knoten aufwärts zur Wurzel zu traversieren. Ohne die Aktualisierung würde `balance()` einem veralteten Zeiger folgen und die Rebalancierung unvollständig oder fehlerhaft ausführen.

Warum beginnt die Rebalancierung mit `balance(parent)`, nicht mit `balance(node)`?

Das Löschen verändert die Höhe des Teilbaums, in dem der Knoten stand. Diese Höhenänderung kann den Balance-Faktor des *Elternknotens* (und aller Vorfahren bis zur Wurzel) aus dem Gleichgewicht bringen. Der gelöschte Knoten ist entweder entfernt oder durch seinen Nachfolger ersetzt – er ist kein sinnvoller Startpunkt für die Aufwärts-Traversal. `balance(parent)` startet genau dort, wo die Höhenänderung zuerst spürbar ist.

5 Komplexitätsanalyse

5.1 Theoretische Herleitung

5.1.1 AVL-Baum (ausgeglichener BST)

Bei jedem Suchschritt wird genau ein Teilbaum mit (ungefähr) halber Größe weiterverfolgt:

$$T(n) = T\left(\frac{n}{2}\right) + \mathcal{O}(1)$$

Master-Theorem mit $a = 1$, $b = 2$, $f(n) = \mathcal{O}(1)$:

$$\log_b a = \log_2 1 = 0 \implies f(n) = \mathcal{O}(n^{\log_b a}) \implies \text{Fall 2: } T(n) = \mathcal{O}(\log n)$$

5.1.2 BST – Worst Case (sortierte Eingabe)

Der Baum ist eine verkettete Liste; jeder Schritt reduziert das Problem um 1:

$$T(n) = T(n - 1) + \mathcal{O}(1)$$

Das Master-Theorem ist nicht anwendbar ($b > 1$ gefordert). Direkte Entfaltung:

$$T(n) = T(n - 2) + 2c = \dots = T(0) + n \cdot c = \mathcal{O}(n)$$

5.1.3 BST – Average Case (zufällige Eingabe)

Bei zufälliger Einfügereihenfolge teilt die Wurzel den Schlüsselraum im Erwartungswert in zwei ungefähr gleich große Hälften – analog zum Durchschnittsfall bei Quicksort. Daraus folgt eine erwartete Höhe von $\mathcal{O}(\log n)$, empirisch ca. $2,5 \cdot \log_2 n$.

5.2 Empirische Messung

Gemessen wird die Anzahl der **Schlüsselvergleiche** beim Suchen des größten eingefügten Schlüssels (Worst-Case-Pfad im jeweiligen Baum) für drei Baumvarianten und drei Eingabegrößen n .

n	BST zufällig	BST sortiert	AVL sortiert
10	6	20	8
100	10	200	14
500	11	1000	18

Stimmen die Messwerte mit den theoretischen Komplexitätsklassen überein?

- **BST sortiert** wächst linear mit n ($2n$ Vergleiche für das letzte Element), was die hergeleitete $\mathcal{O}(n)$ -Komplexität bestätigt.
- **BST zufällig** und **AVL sortiert** wachsen logarithmisch: Für $n = 500$ liegen die Vergleiche bei ca. 11 bzw. 18 – weit unter dem linearen Worst Case. Das bestätigt $\mathcal{O}(\log n)$ für beide, wobei der AVL-Baum die Schranke *garantiert* und der zufällige BST sie nur im Erwartungswert einhält.